Capítulo 10 – Estrutura de Dados

PPGEC2316 – Algoritmos de Engenharia

Raphael Ramos da Silva

Universidade Federal do Rio Grande do Norte

18 de outubro de 2026

Referência: T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4ª ed., MIT Press, 2022 – Parte III.

Roteiro

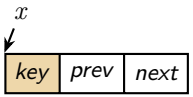
1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Conjuntos dinâmicos

- Conjuntos manipulados por algoritmos podem crescer, encolher ou mudar ao longo do tempo: são conjuntos dinâmicos.
- Um conjunto dinâmico que suporta inserir, remover e testar pertinência é chamado de dicionário.
- Cada elemento é um objeto, acessado por meio de um ponteiro, contendo:
  - uma chave (*key*) identificadora;
  - atributos manipulados pelas operações (p. ex. ponteiros).
- Algumas estruturas supõem chaves de um conjunto totalmente ordenado (permite falar em mínimo, próximo maior, ...).



objeto apontado por x
atributos: $x.key$, $x.prev$, $x.next$

Operações em conjuntos dinâmicos

Consultas (*queries*)

- $\textsc{Search}(S, k)$ retorna um ponteiro x para um elemento com $x.key = k$, ou NIL.
- $\textsc{Minimum}(S)$, $\textsc{Maximum}(S)$ elemento de menor (maior) chave.
- $\textsc{Successor}(S, x)$ próximo elemento maior que x, ou NIL se x é o máximo.
- $\textsc{Predecessor}(S, x)$ próximo elemento menor que x, ou NIL se x é o mínimo.

Operações modificadoras

- $\textsc{Insert}(S, x)$ adiciona ao conjunto o elemento apontado por x.
- $\textsc{Delete}(S, x)$ dado um *ponteiro* x (não uma chave), remove x de S.

Roteiro

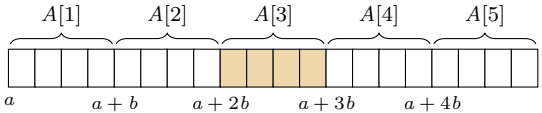
1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Arrays

- Um array é armazenado como uma sequência contígua de bytes na memória.
- Se o primeiro índice é s, o array começa no endereço a e cada elemento ocupa b bytes, o i-ésimo elemento ocupa os bytes

$$a + b(i - s) \quad \text{até} \quad a + b(i - s + 1) - 1.$$

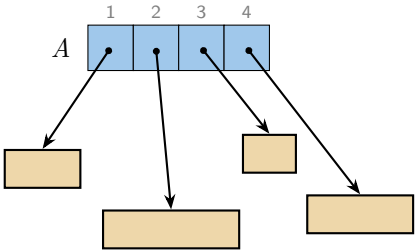
- $s = 1$: bytes $a + b(i-1)$ a $a + bi - 1$; $s = 0$: bytes $a + bi$ a $a + b(i+1) - 1$.
- No modelo RAM, acessar qualquer elemento leva tempo constante, independentemente do índice.



$s = 1$, $b = 4$:
$A[3]$ ocupa os bytes
$a + 2b$ a $a + 3b - 1$

Arrays de elementos de tamanhos diferentes

- A maioria das linguagens exige que todos os elementos de um array tenham o mesmo tamanho b.
- Se os objetos têm tamanhos variáveis, o array guarda ponteiros para os objetos (ponteiros têm tamanho fixo).
- As fórmulas anteriores dão o endereço do *ponteiro*; depois basta segui-lo até o objeto.



objetos de tamanhos variados

Matrizes

- Uma matriz $m \times n$ é normalmente representada por um ou mais arrays unidimensionais.
- Row-major: armazenada linha a linha. Column-major: coluna a coluna.

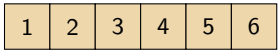
$$M = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \qquad \text{row-major: } 1\,2\,3\,4\,5\,6 \qquad \text{column-major: } 1\,4\,2\,5\,3\,6$$

- Com índices começando em s, o elemento $M[i, j]$ está na posição:

	row-major	column-major
geral	$s + n(i - s) + (j - s)$	$s + m(j - s) + (i - s)$
$s = 1$	$n(i - 1) + j$	$i + m(j - 1)$
$s = 0$	$ni + j$	$i + mj$

- Exemplo ($s = 1$): $M[2, 1]$ está na posição $3(2 - 1) + 1 = 4$ (row-major) e $2 + 2(1 - 1) = 2$ (column-major).

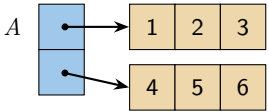
Matrizes: quatro formas de armazenar M



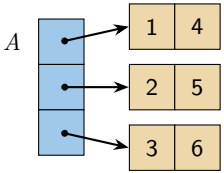
(a) row-major, array único



(b) column-major, array único



(c) row-major, um array por linha
$A[i][j] = M[i, j]$



(d) column-major, um array por coluna
$A[j][i] = M[i, j]$

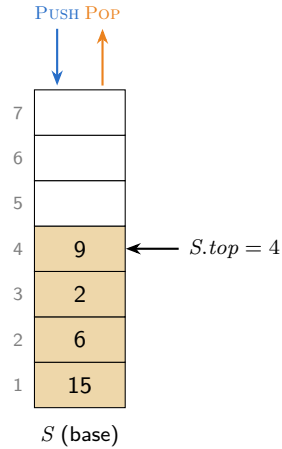
Representações com array único costumam ser mais eficientes; múltiplos arrays são mais flexíveis (permitem *ragged arrays*, linhas de tamanhos diferentes).

Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Stacks (pilhas)

- Conjunto dinâmico em que DELETE remove o elemento inserido mais recentemente: política LIFO (*last-in, first-out*).
- INSERT $\rightarrow$ PUSH; DELETE $\rightarrow$ POP (sem argumento de elemento).
- Implementação com array $S[1:n]$ e atributos:
  - $S.top$: índice do elemento mais recente;
  - $S.size$: tamanho n do array.
- A pilha é $S[1:S.top]$; $S[1]$ é a base e $S[S.top]$ o topo.
- $S.top = 0$: pilha vazia. Pop em pilha vazia: underflow; $S.top$ excedendo $S.size$: overflow.



Stacks: STACK-EMPTY, PUSH e POP

STACK-EMPTY(S)

```
1 if S.top == 0
2     return TRUE
3 else return FALSE
```

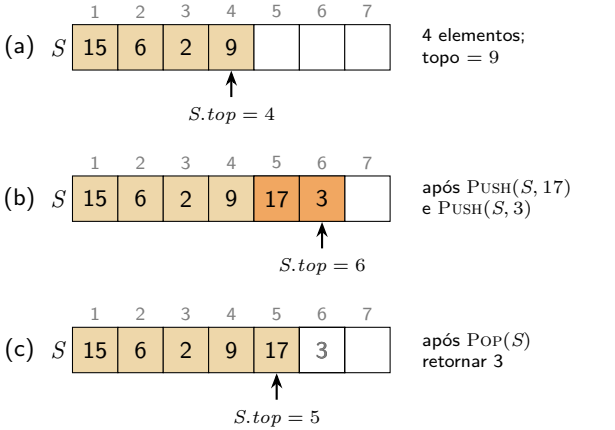
PUSH(S, x)

```
1 if S.top == S.size
2     error "overflow"
3 else S.top = S.top + 1
4     S[S.top] = x
```

POP(S)

```
1 if STACK-EMPTY(S)
2     error "underflow"
3 else S.top = S.top - 1
4     return S[S.top + 1]
```

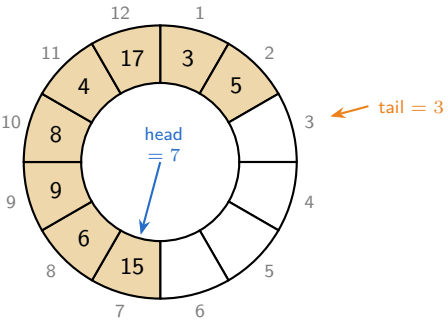
Cada operação executa em $O(1)$.



Apenas as posições em bege pertencem à pilha: o 3 continua no array em (c), mas não está mais na pilha.

Queues (filas)

- DELETE remove o elemento que está há mais tempo no conjunto: política FIFO (*first-in, first-out*).
- INSERT $\rightarrow$ ENQUEUE; DELETE $\rightarrow$ DEQUEUE.
- Array $Q[1:n]$, com $Q.size = n$, armazena até $n-1$ elementos:
  - $Q.head$: índice da cabeça;
  - $Q.tail$: próxima posição livre.
- Elementos em $Q.head, Q.head+1, \ldots, Q.tail-1$, com *wrap around*: a posição 1 segue a posição n.
- Vazia: $Q.head = Q.tail$ (inicialmente ambos $= 1$).
- Cheia: $Q.head = Q.tail+1$, ou $Q.head = 1$ e $Q.tail = Q.size$.



Visão circular da fila $Q[1:12]$.

Queues: Enqueue e Dequeue

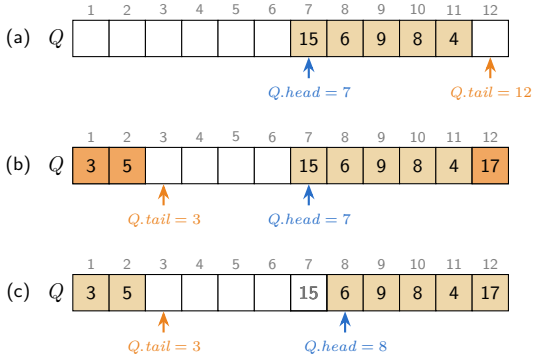
ENQUEUE(Q, x)

1. $Q[Q.tail] = x$
2. **if** $Q.tail == Q.size$
3. $\quad Q.tail = 1$
4. **else** $Q.tail = Q.tail + 1$

DEQUEUE(Q)

1. $x = Q[Q.head]$
2. **if** $Q.head == Q.size$
3. $\quad Q.head = 1$
4. **else** $Q.head = Q.head + 1$
5. **return** x

Cada operação: $O(1)$.

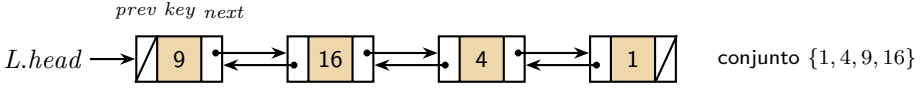


Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Linked lists (listas ligadas)

- Objetos organizados em ordem linear determinada por ponteiros em cada objeto (e não por índices, como no array).
- Lista duplamente ligada L: cada elemento tem key, $next$ e $prev$.
  - $x.prev = \text{NIL}$: x é a cabeça (*head*); $x.next = \text{NIL}$: x é a cauda (*tail*).
  - $L.head$ aponta para o primeiro elemento; $L.head = \text{NIL}$: lista vazia.
- Variações: simplesmente ou duplamente ligada; ordenada ou não; circular ou não.



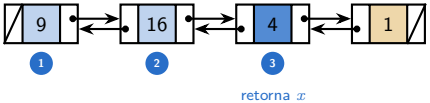
Busca: List-Search

List-Search(L, k)

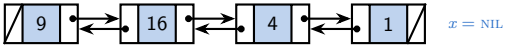
```
1  x = L.head
2  while x ≠ NIL  and x.key ≠ k
3      x = x.next
4  return x
```

- Busca linear pelo primeiro elemento com chave k.
- Pior caso: $\Theta(n)$.

List-Search$(L, 4)$:



List-Search$(L, 7)$:

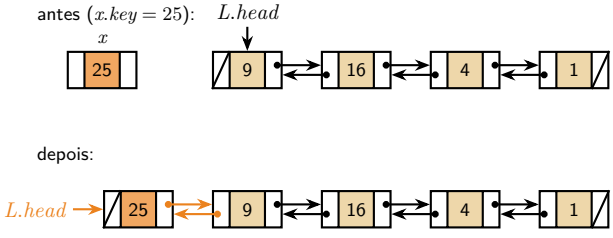


Inserção na cabeça: LIST-PREPEND

LIST-PREPEND(L, x)

1 $x.next = L.head$
2 $x.prev =$ NIL
3 **if** $L.head \neq$ NIL
4 $\quad L.head.prev = x$
5 $L.head = x$

Adiciona x (com $x.key$ já definido) na frente da lista em $O(1)$.



Ponteiros alterados em laranja.

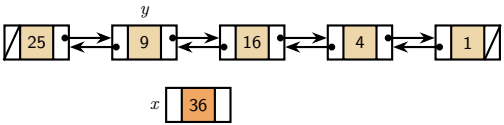
Inserção após um elemento: LIST-INSERT

LIST-INSERT(x, y)

```
1  x.next = y.next
2  x.prev = y
3  if y.next ≠ NIL
4      y.next.prev = x
5  y.next = x
```

"Emenda" x logo após y em $O(1)$. Não referencia a lista L.

antes ($x.key = 36$, y aponta para 9):



depois:



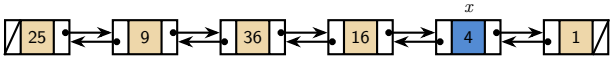
Remoção: List-Delete

List-Delete(L, x)

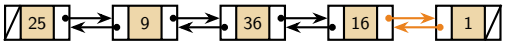
```
1  if x.prev ≠ NIL
2      x.prev.next = x.next
3  else L.head = x.next
4  if x.next ≠ NIL
5      x.next.prev = x.prev
```

- Recebe um *ponteiro* para x: $O(1)$.
- Remover por chave exige List-Search antes: $\Theta(n)$.

antes (x aponta para 4):



depois:

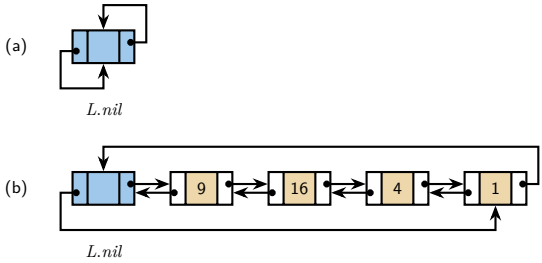




Comparação: inserir/remover o primeiro elemento de um array custa $\Theta(n)$ (deslocamentos), contra $O(1)$ na lista duplamente ligada; já acessar o k-ésimo elemento custa $O(1)$ no array e $\Theta(k)$ na lista.

Sentinelas

- Uma sentinela (L) é um objeto fictício que remove a necessidade das condicionais do LIST-INSERT e LIST-DELETE
- $L.nil$ representa NIL, mas tem todos os atributos dos demais objetos.
- A lista se torna circular, duplamente ligada, com sentinela:
  - $L.nil.next$ aponta para a cabeça;
  - $L.nil.prev$ aponta para a cauda;
  - $L.head$ deixa de ser necessário.
- Lista vazia: só a sentinela, apontando para si mesma.



(a) lista vazia; (b) a lista $\langle 9, 16, 4, 1 \rangle$ com sentinela (azul).

Remoção e inserção com sentinela

LIST-DELETE$'(x)$

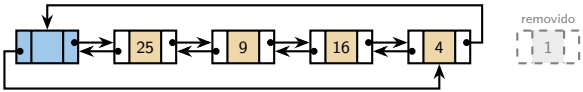
1. $x.prev.next = x.next$
2. $x.next.prev = x.prev$

LIST-INSERT$'(x, y)$

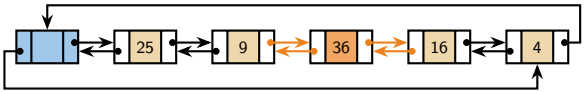
1. $x.next = y.next$
2. $x.prev = y$
3. $y.next.prev = x$
4. $y.next = x$

Inserir na cabeça: $y = L.nil$; na cauda: $y = L.nil.prev$. Nunca remova a sentinela!

LIST-DELETE$'(x)$, x aponta para 1:



LIST-INSERT$'(x, y)$, $x.key = 36$, y aponta para 9:



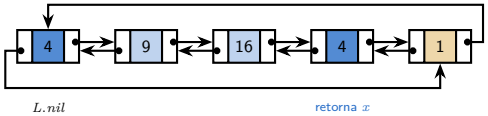
Busca com sentinela: LIST-SEARCH′

LIST-SEARCH′(L, k)

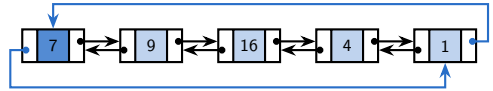
```
1  L.nil.key = k                    // store the key in the sentinel
2  x = L.nil.next                   // start at the head of the list
3  while x.key ≠ k
4      x = x.next
5  if x == L.nil                    // found k in the sentinel
6      return NIL                   // k was not really in the list
7  else return x                    // found k in element x
```

- A chave é garantidamente encontrada (na lista ou na sentinela): uma comparação a menos por iteração.
- Mesmo tempo assintótico, constante menor.

LIST-SEARCH′$(L, 4)$:



LIST-SEARCH′$(L, 7)$:



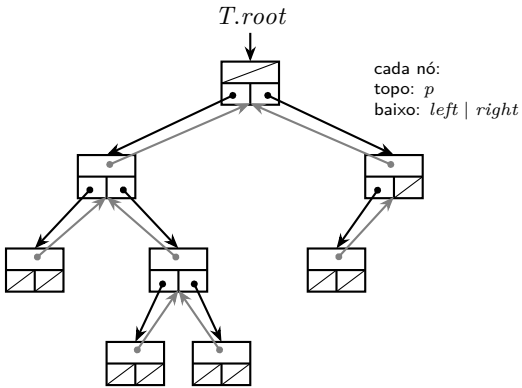
$x = L.nil \Rightarrow$ NIL

Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Árvores binárias

- Cada nó é um objeto com key e ponteiros:
  - p: pai;
  - $left$: filho esquerdo;
  - $right$: filho direito.
- $x.p = \text{NIL}$: x é a raiz.
- $x.left = x.right = \text{NIL}$: x é uma folha.
- $T.root$ aponta para a raiz;
  $T.root = \text{NIL}$: árvore vazia.

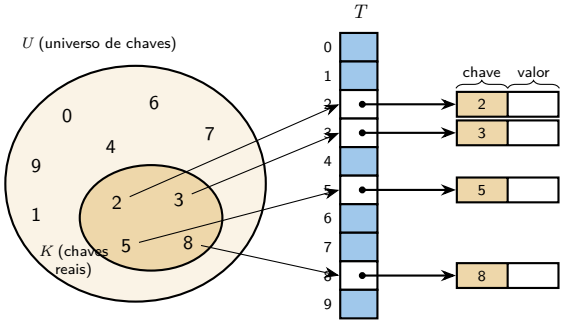


Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Tabelas de endereçamento direto

- Considere um conjunto universo $U = \{0, 1, \ldots, m-1\}$ de chaves.
- Tabela de endereçamento direto $T[0 : m-1]$ mapeia cada posição corresponde a uma chave de U.
- Desvantagem: as chaves $k \in U$ precisam existir na memória, mesmo que elas nunca sejam utilizadas na sua aplicação



Endereçamento direto: operações

DIRECT-ADDRESS-SEARCH(T, k)

1 **return** $T[k]$

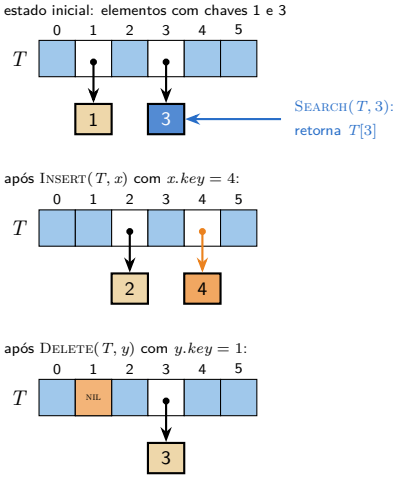
DIRECT-ADDRESS-INSERT(T, x)

1 $T[x.key] = x$

DIRECT-ADDRESS-DELETE(T, x)

1 $T[x.key] = \text{NIL}$

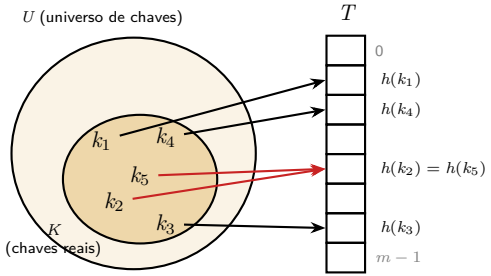
Cada operação: $O(1)$ no pior caso.



- Se U é grande, uma tabela de tamanho $|U|$ é impraticável; e o conjunto K de chaves armazenadas é muito menor que U.
- Uma função hash calcula o slot a partir da chave:

$$h : U \to \{0, 1, \ldots, m-1\}, \qquad m \ll |U|.$$

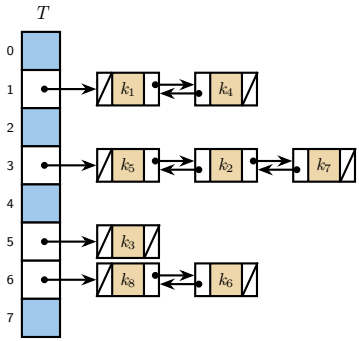
- O elemento de chave k vai para o slot $h(k)$ (*valor hash* de k).
- Armazenamento: $\Theta(|K|)$, busca em $O(1)$ em média.
- Colisão: duas chaves com o mesmo valor hash. Como $|U| > m$, colisões são inevitáveis.



para $h(k) = k \bmod m$, k_2 e k_5 colidem

Encadeamento

- O encadeamento (*chaining*) é uma forma de lidar com colisões
- Cada slot $T[j]$ aponta para uma lista ligada com todos os elementos cujo valor hash é j (NIL se não houver).



$$h(k_1) = h(k_4) = 1$$
$$h(k_5) = h(k_2) = h(k_7) = 3$$

Operações com encadeamento

CHAINED-HASH-INSERT(T, x)

1 LIST-PREPEND$(T[h(x.key)], x)$

CHAINED-HASH-SEARCH(T, k)

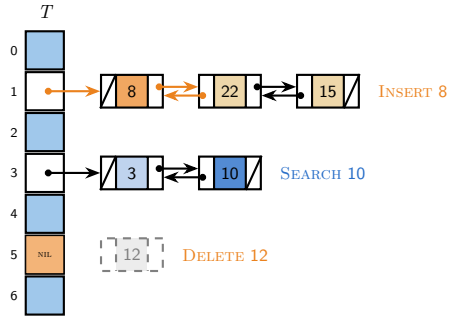
1 **return** LIST-SEARCH$(T[h(k)], k)$

CHAINED-HASH-DELETE(T, x)

1 LIST-DELETE$(T[h(x.key)], x)$

- Insert: $O(1)$ no pior caso (supõe x ausente).
- Search: proporcional ao tamanho da lista.
- Delete: $O(1)$ com listas duplamente ligadas.

$m = 7$, $h(k) = k \bmod 7$, após inserir 15, 22, 12, 10, 3:



Pior caso

Todas as n chaves no mesmo slot: busca $\Theta(n)$, em que n número de elementos armazenados

Caso médio

Seja $\alpha = n/m$ (fator de carga). Se a função hash espalha bem as chaves, cada lista tem, em média, α elementos. Buscar uma chave custa: 1 (calcular o hash e ir ao slot) $+ \alpha$ (percorrer a lista).

- busca *sem sucesso*: percorre a lista inteira $\Rightarrow \Theta(1 + \alpha)$;
- busca *com sucesso*: para no meio da lista, em média $\Rightarrow$ também $\Theta(1 + \alpha)$.

Se $n = O(m)$, então $\alpha = O(1)$: todas as operações de dicionário em $O(1)$ em média.

Método da divisão

$$h(k) = k \bmod m.$$

Ex.: $m = 12$, $k = 100 \Rightarrow h(k) = 4$. Rápido (uma divisão). Funciona bem se m é primo não muito próximo de uma potência de 2 — mas sem garantias.

Método da multiplicação

$$h(k) = \lfloor m\,(kA \bmod 1) \rfloor, \qquad 0 < A < 1,$$

em que $kA \bmod 1 = kA - \lfloor kA \rfloor$ é a parte fracionária de kA. O valor de m não é crítico.

Endereçamento aberto (*open addressing*)

- Todos os elementos ficam na própria tabela: cada slot contém um elemento ou NIL. Sem listas, sem ponteiros $\Rightarrow \alpha \leq 1$.
- Para inserir, sondamos (*probe*) a tabela até achar um slot vazio. A sequência de sondagem depende da chave:

$$h : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}$$

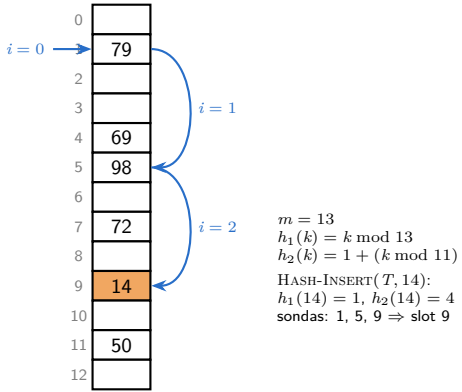
- A sequência de sondagem $\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$ deve ser uma permutação de $\langle 0, 1, \ldots, m-1\rangle$, onde $h(k,i)$ é a i-ésima tentativa (sondagem) para achar a chave k.
- A busca percorre a mesma sequência e para (sem sucesso) ao achar um slot vazio.

Inserção: HASH-INSERT

HASH-INSERT(T, k)

```
1  i = 0
2  repeat
3      q = h(k, i)
4      if T[q] == NIL
5          T[q] = k
6          return q
7      else i = i + 1
8  until i == m
9  error "hash table overflow"
```

Retorna o slot onde k foi armazenada ou sinaliza tabela cheia.

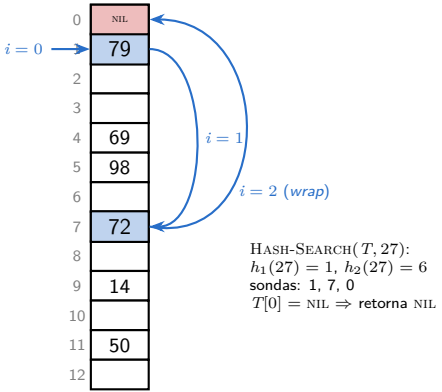


Busca: HASH-SEARCH

HASH-SEARCH(T, k)

```
1  i = 0
2  repeat
3      q = h(k, i)
4      if T[q] == k
5          return q
6      i = i + 1
7  until T[q] == NIL  or i == m
8  return NIL
```

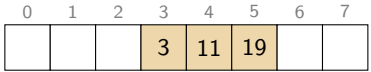
Percorre a mesma sequência da inserção: encontrar um slot vazio prova que k não está na tabela.



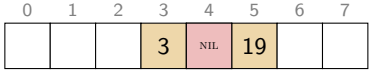
Remoção em endereçamento aberto

- Remover a chave do slot q marcando-o com NIL é um erro: chaves inseridas depois, cuja sondagem passou por q, se tornariam inacessíveis.
- Solução: marcar o slot com o valor especial DELETED.
  - HASH-INSERT trata DELETED como vazio;
  - HASH-SEARCH passa por cima de DELETED.
- Desvantagem: o tempo de busca deixa de depender só de $\alpha \Rightarrow$ encadeamento é preferido quando há muitas remoções.

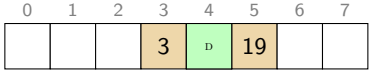
$h(k, i) = (k + i) \bmod 8$



chaves 3, 11, 19 colidem no slot 3



remover 11 com NIL: busca por 19 para no slot 4!



com DELETED: a busca continua e acha 19

Implementação da sondagem no endereçamento aberto

Double hashing

$$h(k, i) = (h_1(k) + i\,h_2(k)) \bmod m$$

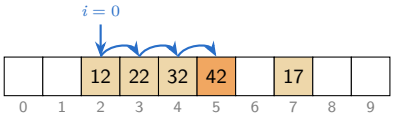
- $h_2(k)$ deve ser primo relativo a m: p. ex. $m = 2^p$ e h_2 ímpar, ou m primo e $0 < h_2(k) < m$.
- Ex.: $h_1(k) = k \bmod m$, $h_2(k) = 1 + (k \bmod m')$, $m' = m - 1$. Com $k = 123456$, $m = 701$, $m' = 700$: $h_1 = 80$, $h_2 = 257$.
- Gera $\Theta(m^2)$ sequências: próximo do ideal.

Linear probing

$h(k, i) = (h_1(k) + i) \bmod m$ (double hashing com $h_2(k) = 1$). Mais simples; apenas m sequências distintas.

Linear probing, $m = 10$, $h_1(k) = k \bmod 10$:

HASH-INSERT$(T, 42)$



Sondas 2, 3, 4 ocupadas; 42 vai para o slot 5.

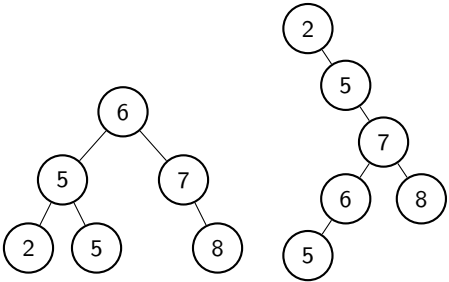
Sequências longas de slots ocupados tendem a crescer.

Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Binary search trees (árvores binárias de busca)

- Suportam SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT e DELETE: servem como dicionário e como fila de prioridade.
- Cada nó: key, dados satélites, $left$, $right$ e p.
- Propriedade de BST: se y está na subárvore esquerda de x, $y.key \leq x.key$; se está na direita, $y.key \geq x.key$.
- Operações em tempo $O(h)$, $h =$ altura: $\Theta(\lg n)$ para árvore completa, mas $\Theta(n)$ se for uma cadeia linear.



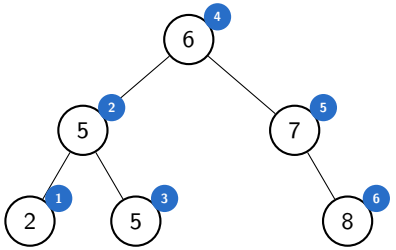
(a) altura 2 (b) altura 4

Duas BSTs com as mesmas chaves.

Percurso em ordem: INORDER-TREE-WALK

```
INORDER-TREE-WALK(x)
1  if x ≠ NIL
2      INORDER-TREE-WALK(x.left)
3      print x.key
4      INORDER-TREE-WALK(x.right)
```

- Imprime a raiz *entre* as subárvores esquerda e direita: chaves em ordem crescente.
- *Preorder*: raiz antes; *postorder*: raiz depois.
- Chamada inicial: INORDER-TREE-WALK($T.root$); tempo $\Theta(n)$.



ordem de impressão (laranja): 2, 5, 5, 6, 7, 8

Busca: TREE-SEARCH

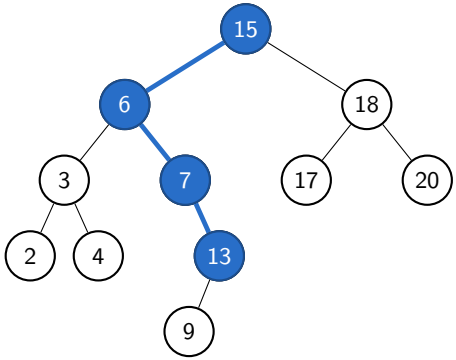
TREE-SEARCH(x, k)

```
1  if x == NIL  or k == x.key
2      return x
3  if k < x.key
4      return TREE-SEARCH(x.left, k)
5  else return TREE-SEARCH(x.right, k)
```

ITERATIVE-TREE-SEARCH(x, k)

```
1  while x ≠ NIL  and k ≠ x.key
2      if k < x.key
3          x = x.left
4      else x = x.right
5  return x
```

Caminho simples descendente: $O(h)$.



TREE-SEARCH$(T.root, 13)$: $15 \to 6 \to 7 \to 13$

Mínimo e máximo: TREE-MINIMUM e TREE-MAXIMUM

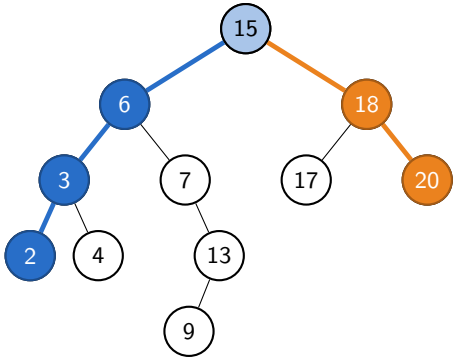
TREE-MINIMUM(x)

```
1  while x.left ≠ NIL
2      x = x.left
3  return x
```

TREE-MAXIMUM(x)

```
1  while x.right ≠ NIL
2      x = x.right
3  return x
```

Seguir ponteiros *left* (ou *right*) a partir da raiz; $O(h)$. Supõe $x \neq$ NIL.

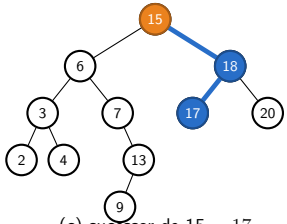


mínimo $= 2$ (azul), máximo $= 20$ (laranja)

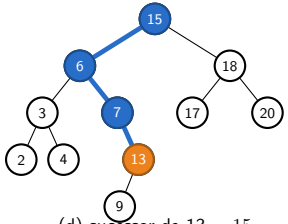
Sucessor: TREE-SUCCESSOR

```
TREE-SUCCESSOR(x)
1  if x.right ≠ NIL
2      return TREE-MINIMUM(x.right)          // leftmost node in right
           subtree
3  else                                      // find the lowest ancestor of x whose
                                             // left child is an ancestor of x
4      y = x.p
5      while y ≠ NIL  and x == y.right
6          x = y
7          y = y.p
8      return y
```

- Sucessor = próximo nó no percurso em ordem; dispensa comparar chaves.
- TREE-PREDECESSOR é simétrico. Ambos $O(h)$.



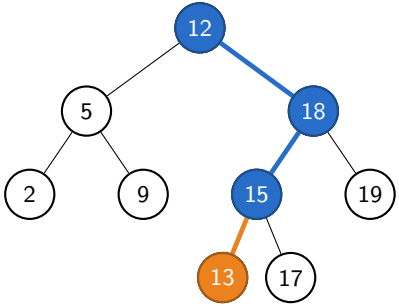
(c) sucessor de 15 = 17
(mínimo da subárvore direita)



(d) sucessor de 13 = 15
(sobe até vir de um filho esquerdo)

Inserção: TREE-INSERT

```
TREE-INSERT(T, z)
 1  x = T.root                    // node being compared with z
 2  y = NIL                       // y will be parent of z
 3  while x ≠ NIL                 // descend until reaching a leaf
 4      y = x
 5      if z.key < x.key
 6          x = x.left
 7      else x = x.right
 8  z.p = y                       // found the location
 9  if y == NIL
10      T.root = z                // tree T was empty
11  elseif z.key < y.key
12      y.left = z
13  else y.right = z
```



Inserção da chave 13: caminho $12 \to 18 \to 15$ (azul); novo nó e ligação ao pai em laranja. O ponteiro "atrasado" y guarda o futuro pai de z. Tempo $O(h)$.

Remoção: estratégia

Remover um nó z de uma BST tem três casos básicos:

1. z não tem filhos: basta substituí-lo por NIL no pai.
2. z tem um filho: o filho "sobe" e ocupa a posição de z.
3. z tem dois filhos: encontre o sucessor y de z (que está na subárvore direita e não tem filho esquerdo) e coloque y na posição de z. Importa se y é ou não o filho direito de z.

O procedimento organiza em quatro casos:

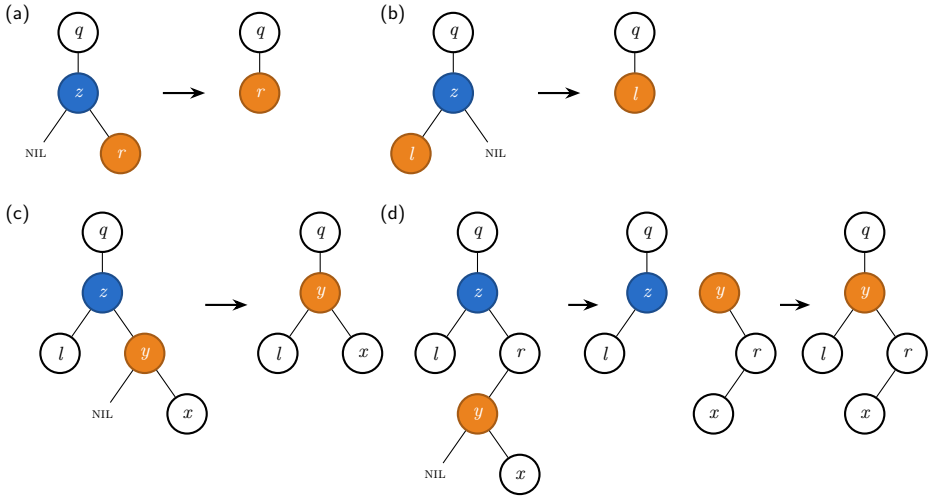
(a) z sem filho esquerdo: substitui z por seu filho direito (talvez NIL);

(b) z só com filho esquerdo: substitui z por esse filho;

(c) dois filhos e o sucessor y é $z.right$: substitui z por y;

(d) dois filhos e $y \neq z.right$: primeiro substitui y por seu filho direito, depois substitui z por y.

Remoção: os quatro casos

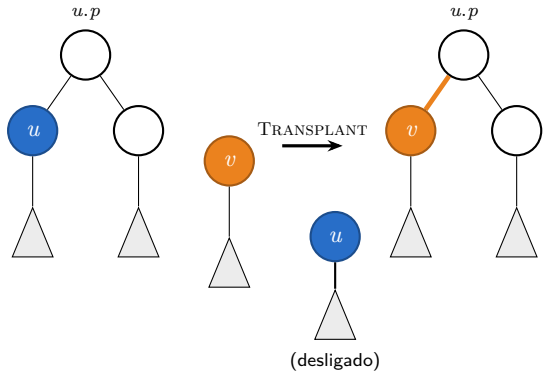


z (azul) é removido; o nó que ocupa sua posição está em laranja. z pode ser a raiz, filho esquerdo ou direito de q.

Sub-rotina TRANSPLANT

```
1  if u.p == NIL
2      T.root = v
3  elseif u == u.p.left
4      u.p.left = v
5  else u.p.right = v
6  if v ≠ NIL
7      v.p = u.p
```

Substitui a subárvore com raiz u pela subárvore com raiz v como filha do pai de u. Não atualiza $v.left$ nem $v.right$ (responsabilidade de quem chama).



Linhas 1–2: u é a raiz; 3–5: u é filho esquerdo ou direito; 6–7: atualiza $v.p$ se $v \neq$ NIL.

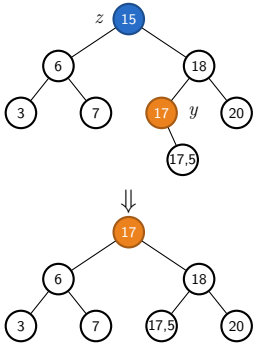
Remoção: TREE-DELETE

TREE-DELETE(T, z)

```
1  if z.left == NIL
2      TRANSPLANT(T, z, z.right)              // replace z by its right child
3  elseif z.right == NIL
4      TRANSPLANT(T, z, z.left)               // replace z by its left child
5  else y = TREE-MINIMUM(z.right)             // y is z's successor
6      if y ≠ z.right                         // is y farther down the tree?
7          TRANSPLANT(T, y, y.right)          // replace y by its right child
8          y.right = z.right                  // z's right child becomes
9          y.right.p = y                      //    y's right child
10     TRANSPLANT(T, z, y)                    // replace z by its successor y
11     y.left = z.left                        // and give z's left child to y,
12     y.left.p = y                           //    which had no left child
```

Tudo é $O(1)$ exceto TREE-MINIMUM: $O(h)$.

TREE-DELETE(T, z), z = nó 15 (caso (d)):



linhas 7–9: 17,5 ocupa o lugar de y; linhas 10–12: y ocupa o lugar de z.

Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Red-black trees (árvores rubro-negras)

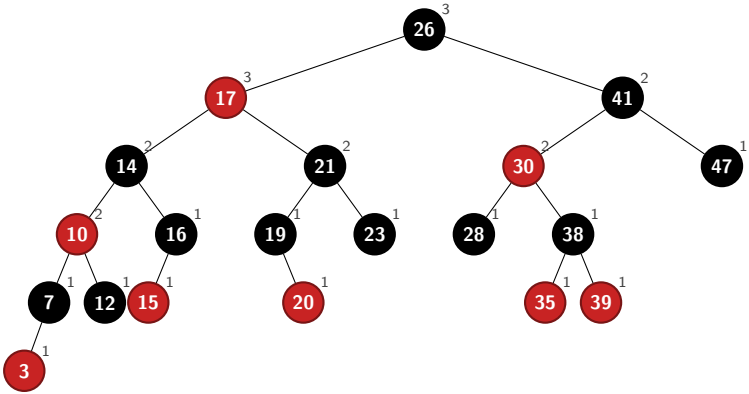
- BSTs balanceadas: garantem $O(\lg n)$ no pior caso.
- Um bit extra por nó: $color \in \{\text{RED}, \text{BLACK}\}$. Atributos: $color$, key, $left$, $right$, p.
- Ponteiros NIL são vistos como folhas (nós externos); nós com chave são internos.

Propriedades rubro-negras

1. Todo nó é vermelho ou preto.
2. A raiz é preta.
3. Toda folha (NIL) é preta.
4. Se um nó é vermelho, ambos os filhos são pretos.
5. Para cada nó, todos os caminhos simples dele até folhas descendentes contêm o mesmo número de nós pretos.

- Nenhum caminho raiz–folha é mais que o dobro de outro: altura $\leq 2\lg(n+1) = O(\lg n)$.

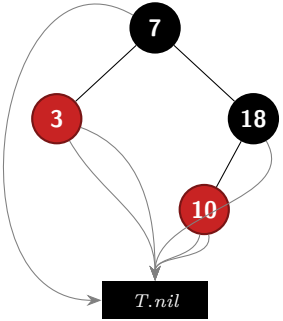
Exemplo de red-black tree



Números em cinza: altura negra $bh(x)$ — nº de nós pretos em qualquer caminho de x (exclusive) até uma folha. As folhas NIL foram omitidas.

A sentinela $T.nil$

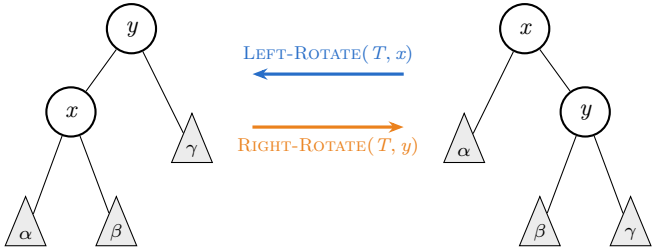
- Para simplificar condições de contorno, uma única sentinela $T.nil$ representa *todas* as folhas NIL e o pai da raiz.
- $T.nil.color = \text{BLACK}$; demais atributos ($p$, $left$, $right$, key) são arbitrários.
- Permite tratar um filho NIL de x como um nó comum cujo pai é x.
- Consequência: SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR rodam em $O(\lg n)$ (com NIL trocado por $T.nil$).
- Mas TREE-INSERT/TREE-DELETE podem violar as propriedades $\Rightarrow$ precisamos de rotações e recolorações (*fixups*).



Todos os ponteiros para NIL (inclusive $T.root.p$) apontam para a sentinela.

Rotações

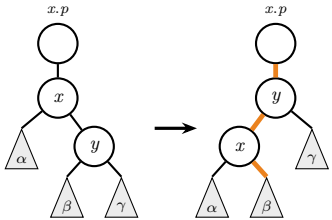
- Operação local que preserva a propriedade de BST, alterando apenas ponteiros, em $O(1)$.
- LEFT-ROTATE(T, x): supõe $x.right = y \neq T.nil$; y vira a nova raiz da subárvore, x vira filho esquerdo de y, e β (filho esquerdo de y) vira filho direito de x.
- RIGHT-ROTATE(T, y) é a inversa.



Left-Rotate

LEFT-ROTATE(T, x)

```
1  y = x.right
2  x.right = y.left          // turn y's left subtree into x's right subtree
3  if y.left ≠ T.nil         // if y's left subtree is not empty ...
4      y.left.p = x          // ...then x becomes the parent of the subtree's root
5  y.p = x.p                 // x's parent becomes y's parent
6  if x.p == T.nil           // if x was the root ...
7      T.root = y            // ...then y becomes the root
8  elseif x == x.p.left      // otherwise, if x was a left child ...
9      x.p.left = y          // ...then y becomes a left child
10 else x.p.right = y        // otherwise, x was a right child
11 y.left = x                // make x become y's left child
12 x.p = y
```



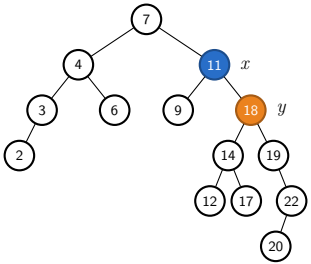
Laranja: ligações alteradas — linhas 2–4: β passa a ser filho de x; 5–10: y assume o lugar de x junto ao pai; 11–12: x vira filho esquerdo de y.

Right-Rotate (simétrico) e exemplo

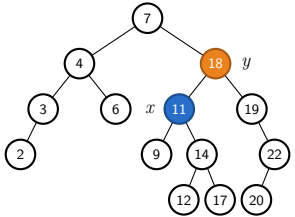
Right-Rotate(T, y)

```
1  x = y.left
2  y.left = x.right
3  if x.right ≠ T.nil
4      x.right.p = y
5  x.p = y.p
6  if y.p == T.nil
7      T.root = x
8  elseif y == y.p.right
9      y.p.right = x
10 else y.p.left = x
11 x.right = y
12 y.p = x
```

Troca-se *left* $\leftrightarrow$ *right* e $x \leftrightarrow y$.



$\Downarrow$ Left-Rotate(T, x)

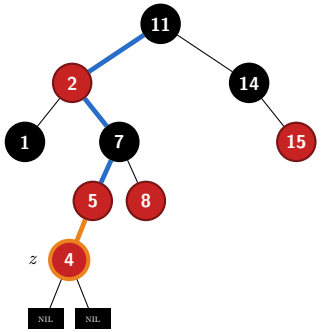


O percurso em ordem é o mesmo antes e depois da rotação.

Inserção: RB-Insert

RB-Insert(T, z)

```
1  x = T.root                          // node being compared with z
2  y = T.nil                           // y will be parent of z
3  while x ≠ T.nil                     // descend until reaching the sentinel
4      y = x
5      if z.key < x.key
6          x = x.left
7      else x = x.right
8  z.p = y                             // found the location
9  if y == T.nil
10     T.root = z                      // tree T was empty
11 elseif z.key < y.key
12     y.left = z
13 else y.right = z
14 z.left = T.nil                      // both of z's children are the sentinel
15 z.right = T.nil
16 z.color = RED                       // the new node starts out red
17 RB-Insert-Fixup(T, z)               // correct any violations
```



Inserção da chave 4: desce como em Tree-Insert (azul), z recebe filhos $T.nil$ e cor vermelha. Como $z.p = 5$ também é vermelho, a propriedade 4 é violada $\Rightarrow$ RB-Insert-Fixup.

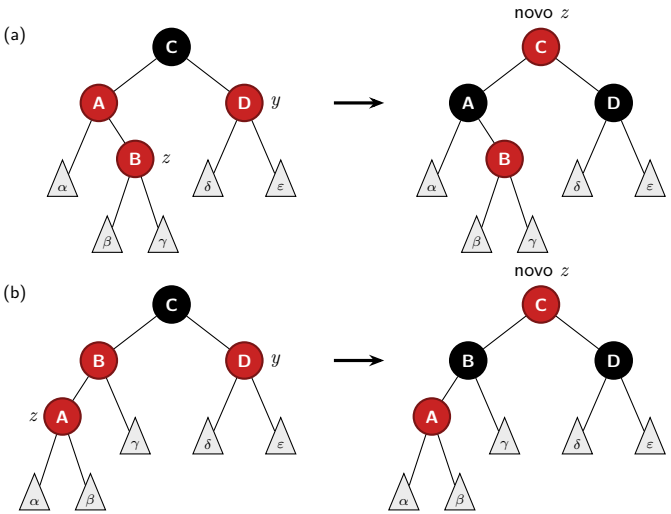
RB-Insert-Fixup

RB-Insert-Fixup(T, z)

```
1  while z.p.color == RED
2      if z.p == z.p.p.left          // is z's parent a left child?
3          y = z.p.p.right                          // y is z's uncle
4          if y.color == RED         // parent and uncle red?
5              z.p.color = BLACK                    // case 1
6              y.color = BLACK                      // case 1
7              z.p.p.color = RED                    // case 1
8              z = z.p.p                            // case 1
9          else
10             if z == z.p.right                    // case 2
11                 z = z.p                          // case 2
12                 LEFT-ROTATE(T, z)                // case 2
13             z.p.color = BLACK                    // case 3
14             z.p.p.color = RED                    // case 3
15             RIGHT-ROTATE(T, z.p.p)               // case 3
16     else       // same as lines 3–15, but with "right"
           and "left" exchanged
17         y = z.p.p.left
18         if y.color == RED
19             z.p.color = BLACK
20             y.color = BLACK
21             z.p.p.color = RED
22             z = z.p.p
23         else
24             if z == z.p.left
25                 z = z.p
26                 RIGHT-ROTATE(T, z)
27             z.p.color = BLACK
28             z.p.p.color = RED
29             LEFT-ROTATE(T, z.p.p)
30 T.root.color = BLACK
```

Só as propriedades 2 (raiz vermelha) e 4 (vermelho com filho vermelho) podem ser violadas. O laço só se repete no caso 1 (z sobe dois níveis): $O(\lg n)$ e no máximo 2 rotações.

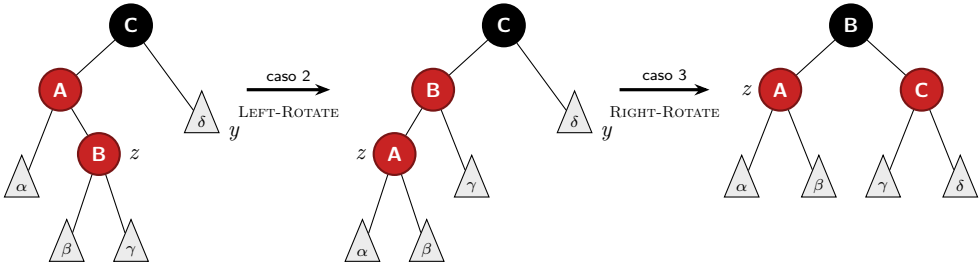
RB-Insert-Fixup: caso 1 — o tio y é vermelho



Linhas 5–8: o avô $z.p.p$ (preto) transfere sua "negritude" para o pai e o tio; o avô fica vermelho e o laço continua com $z = z.p.p$.

Vale tanto se z é filho direito (a) quanto esquerdo (b). Subárvores $\alpha, \ldots, \varepsilon$ têm raiz preta e mesma altura negra.

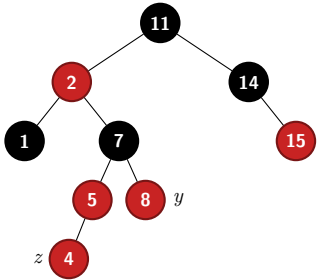
RB-Insert-Fixup: casos 2 e 3 — o tio y é preto



- **Caso 2** (z é filho direito, linhas 10–12): $z = z.p$ e rotação à esquerda: transforma no caso 3 sem alterar alturas negras.
- **Caso 3** (z é filho esquerdo, linhas 13–15): recolore pai (preto) e avô (vermelho) e rotaciona o avô à direita. Não há mais dois vermelhos seguidos: o laço termina.

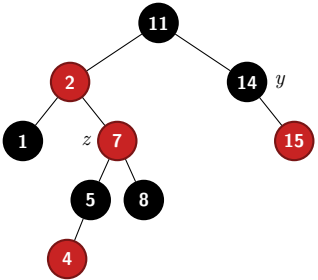
RB-Insert-Fixup: exemplo completo (1/2)

(a)



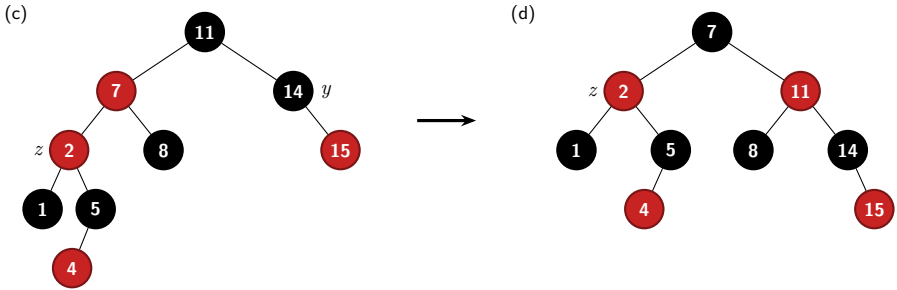
z e $z.p$ vermelhos; tio y vermelho
$\Rightarrow$ caso 1

(b)



$z = 7$ é filho direito; tio $y = 14$ preto
$\Rightarrow$ caso 2

RB-Insert-Fixup: exemplo completo (2/2)



após Left-Rotate$(T, 2)$:
$z = 2$ é filho esquerdo $\Rightarrow$ caso 3

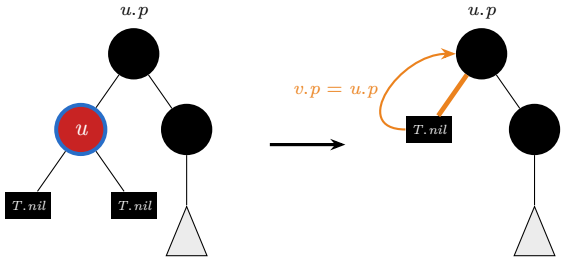
recoloração e Right-Rotate$(T, 11)$:
árvore rubro-negra válida

Remoção: RB-Transplant

RB-Transplant(T, u, v)

```
1  if u.p == T.nil
2      T.root = v
3  elseif u == u.p.left
4      u.p.left = v
5  else u.p.right = v
6  v.p = u.p
```

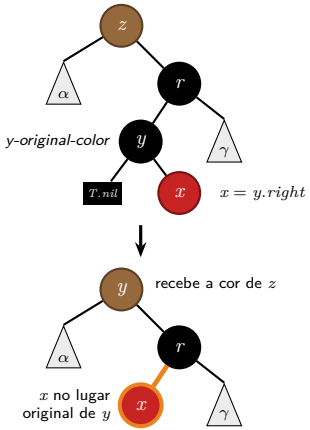
Diferenças para Transplant: usa $T.nil$ em vez de NIL e a atribuição a $v.p$ é incondicional — vale mesmo se $v = T.nil$.



Exemplo com $v = T.nil$: a sentinela passa a ter $p = u.p$, informação usada depois por RB-Delete-Fixup.

Remoção: RB-Delete

```
RB-Delete(T, z)
 1  y = z
 2  y-original-color = y.color
 3  if z.left == T.nil
 4      x = z.right
 5      RB-Transplant(T, z, z.right)          // replace z by its right child
 6  elseif z.right == T.nil
 7      x = z.left
 8      RB-Transplant(T, z, z.left)           // replace z by its left child
 9  else y = Tree-Minimum(z.right)            // y is z's successor
10      y-original-color = y.color
11      x = y.right
12      if y ≠ z.right                        // is y farther down the tree?
13          RB-Transplant(T, y, y.right)      // replace y by its right child
14          y.right = z.right                 // z's right child becomes
15          y.right.p = y                     //    y's right child
16      else x.p = y                          // in case x is T.nil
17      RB-Transplant(T, z, y)                // replace z by its successor y
18      y.left = z.left                       // and give z's left child to y,
19      y.left.p = y                          //    which had no left child
20      y.color = z.color
21  if y-original-color == black              // if any violations occurred,
22      RB-Delete-Fixup(T, x)                 //    correct them
```



y é o nó removido ou movido; x ocupa sua posição. Se y era preto, x ganha um "preto extra" e RB-Delete-Fixup é chamado.

RB-Delete-Fixup (linhas 1–22: x é filho esquerdo)

RB-Delete-Fixup(T, x)

```
1  while x ≠ T.root and x.color == BLACK
2      if x == x.p.left                          // is x a left child?
3          w = x.p.right                         // w is x's sibling
4          if w.color == RED
5              w.color = BLACK                   // case 1
6              x.p.color = RED                   // case 1
7              LEFT-ROTATE(T, x.p)               // case 1
8              w = x.p.right                     // case 1
9          if w.left.color == BLACK and
              w.right.color == BLACK
10             w.color = RED                     // case 2
11             x = x.p                           // case 2
12     else
13         if w.right.color == BLACK
14             w.left.color = BLACK              // case 3
15             w.color = RED                     // case 3
16             RIGHT-ROTATE(T, w)                // case 3
17             w = x.p.right                     // case 3
18         w.color = x.p.color                   // case 4
19         x.p.color = BLACK                     // case 4
20         w.right.color = BLACK                 // case 4
21         LEFT-ROTATE(T, x.p)                   // case 4
22         x = T.root                            // case 4
```

Dentro do laço, x é sempre um nó não raiz duplamente preto, e w (irmão de x) não pode ser $T.nil$.

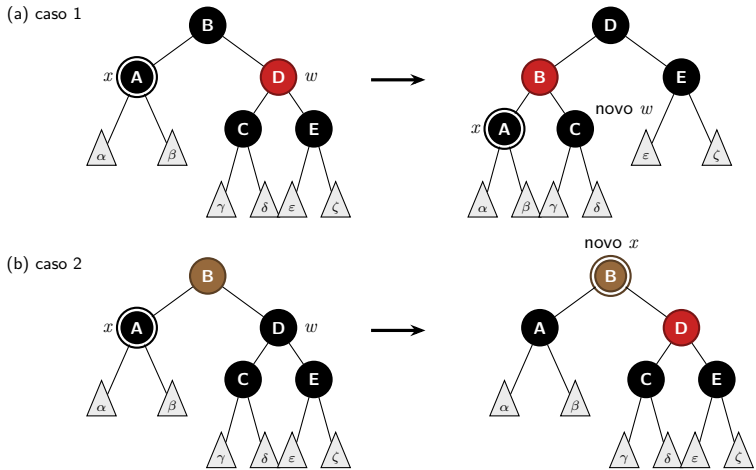
RB-Delete-Fixup (linhas 23–44: x é filho direito)

RB-Delete-Fixup(T, x) (cont.)

```
23      else    // same as lines 3–22, but with "right" and
                "left" exchanged
24          w = x.p.left
25          if w.color == RED
26              w.color = BLACK
27              x.p.color = RED
28              Right-Rotate(T, x.p)
29              w = x.p.left
30          if w.right.color == BLACK  and
             w.left.color == BLACK
31              w.color = RED
32              x = x.p
33      else
34          if w.left.color == BLACK
35              w.right.color = BLACK
36              w.color = RED
37              Left-Rotate(T, w)
38              w = x.p.left
39          w.color = x.p.color
40          x.p.color = BLACK
41          w.left.color = BLACK
42          Right-Rotate(T, x.p)
43          x = T.root
44  x.color = BLACK
```

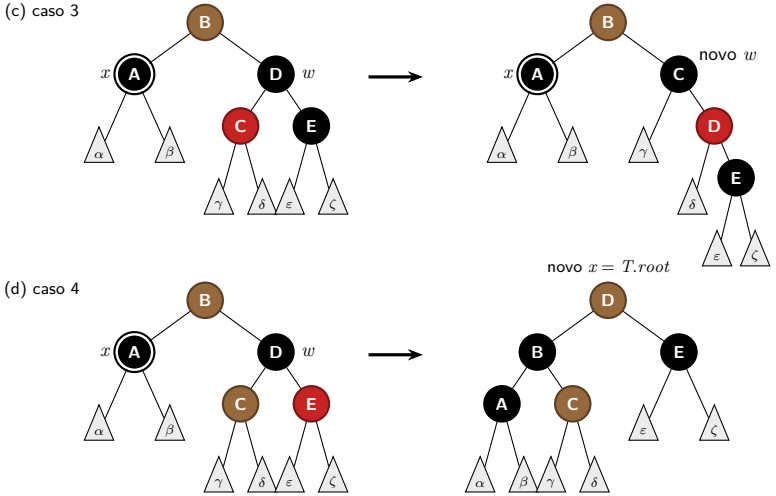
Só o caso 2 repete o laço (x sobe, sem rotações). Casos 1, 3 e 4 terminam após $O(1)$ recolorações e no máximo 3 rotações: RB-Delete é $O(\lg n)$.

RB-Delete-Fixup: casos 1 e 2



Caso 1 (w vermelho): troca cores de w e $x.p$ e rotaciona $x.p$ à esquerda $\rightarrow$ casos 2, 3 ou 4. Caso 2 (w preto, filhos pretos): tira um preto de x e de w e o passa para $x.p$ (x sobe). Nó marrom: cor c qualquer; borda dupla: preto extra.

RB-Delete-Fixup: casos 3 e 4



Caso 3 (w preto, $w.left$ vermelho, $w.right$ preto): troca cores de w e $w.left$ e rotaciona w à direita $\rightarrow$ caso 4. Caso 4 (w preto, $w.right$ vermelho): recolore, rotaciona $x.p$ à esquerda e o preto extra desaparece; o laço termina.

Roteiro

1. Introdução: conjuntos dinâmicos
2. Arrays e matrizes
3. Stacks e queues
4. Linked lists
5. Representação de árvores enraizadas
6. Hash tables
7. Binary search trees
8. Red-black trees
9. Fechamento

Resumo das complexidades

Estrutura	SEARCH	INSERT	DELETE	MIN/MAX	SUCC/PRED
Array não ordenado	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Array ordenado	$O(\lg n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Stack / queue	—	$O(1)$	$O(1)$	—	—
Lista duplamente ligada	$\Theta(n)$	$O(1)$	$O(1)$	$\Theta(n)$	$\Theta(n)$
Hash table (encadeamento)	$\Theta(1+\alpha)$ médio	$O(1)$	$O(1)$	—	—
Hash table (end. aberto)	$\leq \frac{1}{1-\alpha}$ sondas	$\leq \frac{1}{1-\alpha}$ sondas	—	—	—
Binary search tree	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(h)$
Red-black tree	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$